



Department of Engineering Cybernetics

TTK4235 - Tilpassede Datasystemer

Heislabb Gruppe 46

Erik Bohne og Lars Lien

March 27, 2025

Contents

1	Introduksjon	2
1.1	Kort beskrivelse av prosjektet	2
1.2	Kort om heismodellen og problemstillingen	2
1.3	Kort om bruk av V-modellen i utviklingsfasen	2
2	Arkitekturdesign	4
2.1	Overordnet struktur	4
2.2	Behandling av bestillinger	5
2.3	Håndtering av data	5
3	Moduldesign	6
3.1	Startup modul	6
3.2	Input modul	6
3.3	State moduler	6
4	Implementasjon	7
4.1	Generell struktur	7
4.2	Implementasjon og testing av hver enkelt modul	8
5	Refleksjon og vurdering	9
5.1	Valg tatt i arkitekturfasen: Vellykket eller ikke?	9
5.2	Har V-modellen gjort prosjektet lettere?	9
5.3	Forbedringer og aspekter ved robusthet	9
5.4	Bruk av KI	10
5.4.1	Effekter av KI-bruk på prosjektet	10
6	Konklusjon og oppsummering	11
A	Appendiks	12
A.1	Doxygen-dokumentasjon	12
A.1.1	Datastrukturer	12
A.1.2	Tilstandsmodul (states.h)	13
A.1.3	Inputmodul (input.h)	14
A.1.4	Oppstartsmodul (startup.h)	14
A.1.5	Grensesnitt mot heismaskinvare (elevio.h)	15
A.1.6	Hovedprogram (main.c)	16
A.2	Tilleggsdiagrammer	17
A.2.1	Tilstandsdiagram	17
A.2.2	Sekvensdiagram	18

Chapter 1

Introduksjon

1.1 Kort beskrivelse av prosjektet

Dette prosjektet omfatter design, implementasjon og testing av et styresystem for en heis. Oppgaven går ut på å lage et program som kontrollerer en heismodell på en robust og pålitelig måte. Programmet må håndtere bevegelse mellom etasjer, bestillinger fra både inne i heisen og i hver etasje, samt ulike feilsituasjoner som kan oppstå under drift.

Prosjektet er gjennomført som en del av emnet TTK4235 - Tilpassede Datasystemer ved NTNU, der vi har benyttet C-programmering for å lage et heiskontrollsystem som kommuniserer med heismaskinvaren via en driver.

1.2 Kort om heismodellen og problemstillingen

Heismodellen i prosjektet er en enkel simulert heis med fire etasjer. Hver etasje har knapper for å bestille heisen opp eller ned (bortsett fra øverste og nederste etasje som bare har én retningsknapp hver), og inne i heisen finnes knapper for å velge destinasjonsetasje. I tillegg finnes det en stopknapp og en sensor som detekterer om noe blokkerer heisdøren.

Hovedproblemstillingen i prosjektet er å utvikle et styresystem som effektivt håndterer alle bestillinger og feilsituasjoner på en måte som gir brukeren en intuitiv og pålitelig opplevelse. Systemet må ha kontroll på heisens posisjon, håndtere køen av bestillinger effektivt, og reagere korrekt på hendelser som stopknapp, obstruksjon i døren, og strømbrudd.

1.3 Kort om bruk av V-modellen i utviklingsfasen

I dette prosjektet har vi benyttet en pragmatisk versjon av V-modellen som rammeverk for utviklingsprosessen. V-modellen illustrerer hvordan utviklingsfaser korresponderer med tilhørende testfaser, og danner en systematisk tilnærming til systemutvikling.

På venstre side av V-en har vi nedbrytingsfasene:

- Systemkrav og spesifikasjoner
- Arkitekturdesign
- Moduldesign

- Implementasjon

På høyre side av V-en har vi testfasene:

- Enhetstesting
- Integrasjonstesting
- Systemtesting

Ved å følge denne modellen har vi først brutt ned systemet i mindre komponenter og deretter implementert og testet hver komponent før de ble integrert til et helhetlig system. Dette har gitt oss en strukturert tilnærming til utviklingen og gjort det enklere å sikre at alle krav blir oppfylt.

I senere kapitler vil vi diskutere hvordan denne modellen har påvirket våre valg og reflektere over hvor godt den har fungert for dette prosjektet.

Chapter 2

Arkitekturdesign

Arkitekturfasen er grunnlaget for hele heisprosjektet, hvor hovedoppgaven var å definere systemets overordnede struktur og organisering. I denne fasen måtte vi ta avgjørende valg som ville påvirke hele implementasjonen. Vi la vekt på å skape en arkitektur som var robust, oversiktlig og enkel å vedlikeholde.

2.1 Overordnet struktur

Den første og viktigste delen av arbeidet gikk ut på å finne en struktur vi mente ville føre til en ryddig og enkel implementasjon av alle modulene. Ved å lese kravspesifikasjonen nøye og se på den grunnleggende virkemåten til heisen, kom vi frem til at en tilstandsmaskin (finite-state-machine) ville være en god løsning.

En av hovedfordelene med tilstandsmaskinen er at vi kunne sette opp et tilstandsdiagram som ga en veldig god oversikt over systemet. Ved å legge til transisjoner og tilstander kunne vi bygge opp tilstandsmaskinen til vi så at alle krav kunne tilfredsstilles med denne typen arkitektur. Dette gjorde at vi kunne oppdage eventuelle mangler i designet før vi begynte å implementere kode, noe som ga en trygghet om at arkitekturen ville fungere som tiltenkt. Figur 2.1 viser systemets tilstander og transisjonene mellom disse.

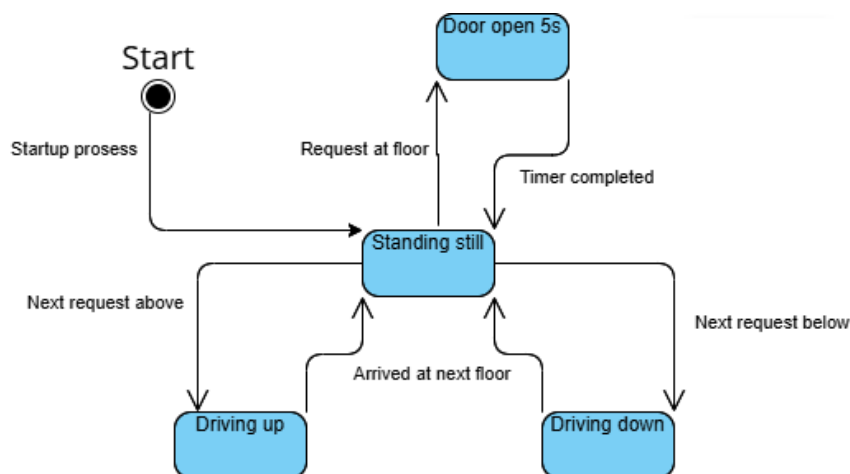


Figure 2.1: Forenklet UML-diagram av heisens tilstandsmaskin

2.2 Behandling av bestillinger

Den neste problemstillingen handlet om hvordan vi skulle lagre og behandle bestillingene fra heispanelet. I utgangspunktet sto valget mellom et dynamisk køsystem og et regelbasert system hvor heisen i hver etasje tar en beslutning om hvor den skal kjøre. Det dynamiske køsystemet virket komplekst uten å gi noen stor gevinst, særlig fordi nye bestillinger fører til en reorganisering av hele køen.

Det regelbaserte systemet vi valgte fungerer ved å ha kontroll på hvilken vei heisen kjører og hvilke etasjer som har bestillinger. Systemet følger disse hovedprinsippene:

- Heisen fortsetter i samme retning så lenge det finnes bestillinger i den retningen
- Den betjener alle bestillinger i gjeldende retning før den endrer retning
- Bestillinger i motsatt retning ignoreres midlertidig og tas med på returen

2.3 Håndtering av data

Siste steg i arkitekturfasen var å finne ut hvordan vi skulle sende data rundt i applikasjonen. Mer spesifikt handler dette om heisens retning, hvilke knapper som er trykket og hvilken tilstand maskinen er i.

Dette ble løst ved å oppbevare all data i en struktur i hovedfilen vist i 2.1. Ingen av tilstandsmodulene lagrer informasjon mellom hver iterasjon av programmet. De får i stedet overlevert systemets nåværende tilstand som en parameter og returnerer den nye tilstanden.

Listing 2.1: Datastrukturen som sendes til hver tilstandsfunksjon

```
1 typedef struct state_data {  
2     State state;           // Nåværende tilstand  
3     int btnStates[4][3];  // Knappetrykk i hver etasje og retning  
4     Direction dir;       // Nåværende retning  
5 } state_data;
```

Chapter 3

Moduldesign

3.1 Startup modul

Startupmodulen har som oppgave å sende heisen til en definert posisjon ved oppstart. Den ignorerer forespørsler og kjører ned til den kommer til en etasje. Modulen kjører bare ved oppstart uavhengig av resten av programmet og bvar derfor lett å teste.

3.2 Input modul

Input modulen kjører en gang hver syklus og har som jobb behandle heislys og knapppanelet med unntak av stop og obstruksjonsknappen. Den første funksjonen lagrer nye knappetrykk i data strukturen og setter lys. Den andre funksjonen kalles når en etasje er besøkt. Den fjerner knappen fra data strukturen og slukker tilsvarende lys.

Den store fordelene med denne modulen er at den abstraherer vekk logikken med å endre knappetaus. Ved å bruke funksjonene vil alltid etasjelys og data stemme overens og sjansen for å gjøre en feil i de andre modulene blir kraftig redusert.

3.3 State moduler

De største delen av programmet er de ulike state modulene. Oversikten over disse modulene og hvordan de hanger sammen kan du se i Figur 2.1 På generelt grunnlag består implementasjonen i hovedsak av if statments som sammsvarer til spesifikasjonene fra oppgaven.

Chapter 4

Implementasjon

Etter å ha definert arkitekturen og moduldesignen for heissystemet, gikk vi videre til implementasjonsfasen. Denne fasen innebar å omsette designet til fungerende kode og teste hver enkelt modul underveis for å sikre korrekt funksjonalitet.

4.1 Generell struktur

Implementasjonen av heissystemet følger tilstandsmaskinarkitekturen som ble definert i arkitekturfasen. Hovedstrukturen består av en hovedløkke som kontinuerlig sjekker nåværende tilstand og kaller riktig tilstandsfunksjon. Hver tilstandsfunksjon utfører handlinger basert på systemets nåværende tilstand og returnerer den neste tilstanden.

Listing 4.1: Hovedløkken i main-filen

```
1  int main() {
2      state_data sData = {STANDING_STILL, {0}, CURRENT_DIR_DOWN};
3
4      elevator_startup();
5
6      while(1){
7          //Floor sensor logic
8          floor = elevio_floorSensor();
9          if(floor >= 0){
10             elevio_floorIndicator(floor);
11         }
12
13         checkForInput(&sData);
14
15         switch (sData.state) {
16
17             case STANDING_STILL:
18                 standing_still(&sData, floor);
19                 break;
20
21             case DRIVING_UP:
22                 driving(&sData, floor, DIRN_UP);
23                 break;
24
25             case DRIVING_DOWN:
26                 driving(&sData, floor, DIRN_DOWN);
27                 break;
28         }
```



```
29     case DOOR_OPEN:
30         door_open_state(&sData, floor);
31         break;
32
33     default:
34         printf("ERROR: Invalid state. ");
35         return -1;
36         break;
37 }
38
39     nanosleep(&(struct timespec){0, 20*1000*1000}, NULL);
40
41 }
42 return 0;
43 }
```

Denne implementasjonen sørger for at heisen alltid er i en veldefinert tilstand, og at overganger mellom tilstander håndteres på en konsistent måte. Den sentraliserte datastrukturen `state_data` fungerer som informasjonsbærer mellom tilstandene og inneholder all nødvendig informasjon for at hver tilstandsfunksjon kan ta riktige beslutninger.

4.2 Implementasjon og testing av hver enkelt modul

Under implementasjonsfasen fulgte vi en inkrementell tilnærming der hver modul ble testet før integrasjon med resten av systemet. Dette ga oss mulighet til å identifisere og korrigere feil tidlig i utviklingsprosessen. For hver tilstandsmodul kunne vi teste funksjonaliteten på den virkelige heisen slik at vi med høy tillit til hver enkelt modul kunne gå videre til integrasjonsfasen. Tilsvarende testet vi inputmodulen med simulerte knappetrykk for å sikre at bestillinger ble riktig registrert og at lys ble riktig aktivert.

Chapter 5

Refleksjon og vurdering

5.1 Valg tatt i arkitekturfasen: Vellykket eller ikke?

Valgene vi tok i arkitekturfasen vise seg stort sett å være gode for implementasjonen av prosjektet. Tilstandsmaskinen var en stor suksess og regelbasert behandling av bestillinger fungerte godt.

Vi måtte tilbake å gjøre endringer på designet når vi implementerte stop og obstruksjonsfunksjonaliteten. Vi fant blant annet ut at det ville være nyttig med et "door open" state, noe vi først ikke hadde forutsett. Alt i alt viste dette hvordan den iterative prosessen i V-modellen ga oss mulighet til å forbedre designet basert på innsikt fra implementasjonsfasen, selv om det betydde at vi måtte justere vår opprinnelige UML-modell.

5.2 Har V-modellen gjort prosjektet lettere?

V-modellen fungerte som et strukturert rammeverk for utviklingsprosessen vår, og bidro til en systematisk tilnærming til prosjektet. Av tidligere erfaring er det lett å bli overveldet av størrelsen på et problem, men ved å strukturert bryte det ned i moduler blir jobben mye mer overkommelig.

En utfordring med V-modellen var at den kan virke noe rigid for et mindre prosjekt som dette. Design og planleggingsfasen var svært nyttige for implementasjonsfasen, men rigid enhetstesting synes vi ikke var spesielt nyttig.

V-modellen påvirket de ulike fasene i prosjektet på følgende måter:

- **Startfasen:** Bruken av UML hjalp oss med å visualisere systemet på et høyt nivå før vi begynte å kode, noe som ga oss en klar retning.
- **Midtfasen:** Moduloppdelingen fra designfasen gjorde det enkelt å fordele arbeid og implementere hver modul individuelt.
- **Sluttfasen:** Testingen ble enklere fordi vi hadde klare forventninger til hver moduls funksjonalitet basert på designet.

5.3 Forbedringer og aspekter ved robusthet

En svakhet ved vår implementasjon er at deler av koden ble noe rotete, spesielt i tilstandsmodulene. Dette skyldes delvis at vi måtte gjøre endringer i designet underveis, som resulterte i at koden ikke alltid fulgte det opprinnelige designet.

Med tanke på robusthet, skalerbarhet og vedlikehold, har V-modellen og UML gitt oss noen fordeler:

- **Robusthet:** Ved å tvinge oss til å tenke gjennom tilstandsoverganger og feilhåndtering på forhånd, hjalp UML-diagrammet oss med å bygge et mer robust system som takler uventede hendelser.
- **Vedlikehold:** Den modulære strukturen gjør det enklere å forstå og endre individuelle deler av systemet uten å påvirke hele kodebasen.

En utfordring vi opplevde var at den strenge strukturen i vår tilnærming gjorde det vanskeligere å implementere nye funksjonaliteter som ikke var planlagt i det opprinnelige designet, som for eksempel håndtering av obstruksjoner i døren.

5.4 Bruk av KI

I løpet av prosjektet eksperimenterte vi med bruk av KI-verktøy, spesielt store språkmodeller som ChatGPT og Claude, som støtte i utviklingsprosessen. Vi anvendte KI-verktøy primært i følgende områder:

Kodegenerering: For standardfunksjonalitet, modulimplementasjon og boilerplate-kode brukte vi KI til å generere utkast som vi deretter tilpasset, testet og forbedret. Dette reduserte tid brukt på rutineoppgaver.

Dokumentasjon: KI ble brukt til å assistere med generering av kommentarer og dokumentasjon, inkludert bidrag til rapporten.

5.4.1 Effekter av KI-bruk på prosjektet

Bruken av KI-verktøy hadde flere effekter på arbeidet vårt:

- **Positive effekter:** Verktøyene hjalp oss med å spare tid på repetitive oppgaver, ga oss forslag til alternative løsninger, og assisterte med feilsøking når vi satt fast.
- **Utfordringer:** Vi måtte fortsatt tilpasse og kvalitetssikre all generert kode manuelt, siden modellene noen ganger produserte kode som ikke passet med vår arkitektur eller ikke oppfylte våre spesifikke krav.
- **Læringsperspektiv:** Selv om KI-verktøyene genererte utkast, måtte vi fortsatt forstå koden for å kunne tilpasse og feilsøke den, noe som bidro til læring.

Det er viktig å understreke at KI-verktøyene fungerte som assistenter, ikke erstattere. De kritiske beslutningene om arkitektur, design og implementasjon ble tatt av oss basert på vår forståelse av problemet og kravene.

Chapter 6

Konklusjon og oppsummering

I dette prosjektet har vi utviklet et heisstyringssystem med en robust arkitektur basert på en tilstandsmaskin. Vi startet med en grundig analyse av kravspesifikasjonen og designet deretter et system som kunne håndtere alle nødvendige heisoperasjoner og feilsituasjoner på en pålitelig måte.

Avslutningsvis har prosjektet gitt oss verdifull erfaring innen utvikling av innebygde systemer, arkitekturdesign, og programvareutvikling med vekt på robusthet og pålitelighet. De erfaringene og lærdommene vi har tilegnet oss gjennom dette prosjektet vil være direkte overførbare til fremtidige embedded systems-prosjekter, enten de involverer heiser eller andre kontrollsystemer.

Appendix A

Appendiks

Dette appendikset inneholder supplerende dokumentasjon og diagrammer som gir ytterligere detaljer om heisstyringssystemet.

A.1 Doxygen-dokumentasjon

Her presenteres en detaljert dokumentasjon av heissystemets kodestruktur i Doxygen-format.

A.1.1 Datastrukturer

Heissystemet benytter følgende hovedstrukturer:

Listing A.1: Heissystemets tilstander

```
1  /**
2   * @brief Mulige tilstander for heisen
3   */
4  typedef enum {
5      DOOR_OPEN = 0,          /**< Døren er åpen */
6      STANDING_STILL = 1,     /**< Heisen står stille med lukket dør */
7      DRIVING_UP = 2,         /**< Heisen beveger seg oppover */
8      DRIVING_DOWN = 3       /**< Heisen beveger seg nedover */
9  } State;
10
11 /**
12  * @brief Heisens retning
13  */
14  typedef enum {
15      CURRENT_DIR_UP = 0,      /**< Siste retning var oppover */
16      CURRENT_DIR_DOWN = 1,    /**< Siste retning var nedover */
17  } Direction;
18
19 /**
20  * @brief Datastruktur som inneholder heissystemets tilstand
21  */
22  typedef struct state_data {
23      State state;              /**< Nåværende tilstand */
24      int btnStates[4][3];      /**< Knappetrykk for alle etasjer og
25                               knappetyper */
26      Direction dir;           /**< Nåværende/siste retning */
27  } state_data;
```

A.1.2 Tilstandsmodul (states.h)

Modulen inneholder funksjoner for å håndtere de ulike tilstandene i heisen.

Listing A.2: Hovedfunksjoner i tilstandsmodulen

```
1  /**
2   * @brief Håndterer tilstanden hvor heisen står stille
3   *
4   * Når heisen er i denne tilstanden, sjekker den etter
5   * nye bestillinger og bestemmer neste tilstand.
6   *
7   * @param data Peger til heisens tilstandsdata
8   * @param floor Nåværende etasje (-1 hvis mellom etasjer)
9   */
10 void standing_still(state_data *data, int floor);
11
12 /**
13  * @brief Åpner heisdøren
14  *
15  * Aktiverer døråpningslampen og venter i 3 sekunder
16  * før den slår av lampen.
17  *
18  * @param data Peger til heisens tilstandsdata
19  */
20 void open_door(state_data *data);
21
22 /**
23  * @brief Håndterer tilstanden med åpen dør
24  *
25  * Styrer tidtakeren for åpen dør og går tilbake til standing_still
26  * når tidtakeren utløper. Sjekker også for hindring og stoppknapp.
27  *
28  * @param data Peger til heisens tilstandsdata
29  * @param floor Nåværende etasje (-1 hvis mellom etasjer)
30  */
31 void door_open_state(state_data *data, int floor);
32
33 /**
34  * @brief Håndterer kjøretilstanden (opp eller ned)
35  *
36  * Kontrollerer heisbevegelsen og sjekker om den skal stoppe
37  * ved etasjer. Håndterer stoppknappen og overganger til andre
38  * tilstander.
39  *
40  * @param data Peger til heisens tilstandsdata
41  * @param floor Nåværende etasje (-1 hvis mellom etasjer)
42  * @param dir Nåværende bevegelsesretning
43  */
44 void driving(state_data *data, int floor, MotorDirection dir);
45
46 /**
47  * @brief Nullstiller alle bestillinger og slår av alle knappelys
48  *
49  * Brukes når stoppknappen trykkes eller i andre situasjoner
50  * hvor alle bestillinger skal kanselleres.
51  *
52  * @param data Peger til heisens tilstandsdata
53  */
```

```
53 void clear_all_orders(state_data *data);
54
55 /**
56  * @brief Sjekker om det er en hindring i døren
57  *
58  * Håndterer hindring ved å stoppe heisen, åpne døren,
59  * og nullstille alle bestillinger.
60  *
61  * @param data Peker til heisens tilstandsdata
62  * @param floor Nåværende etasje (-1 hvis mellom etasjer)
63  * @return 1 hvis hindring er aktiv, 0 ellers
64  */
65 int check_obstruction(state_data *data, int floor);
```

A.1.3 Inputmodul (input.h)

Modulen håndterer knappetrykk og lyssignaler.

Listing A.3: Hovedfunksjoner i inputmodulen

```
1 /**
2  * @brief Sjekker for knappetrykk og oppdaterer tilstandsdata
3  *
4  * Skanner alle heisknapper (hall opp, hall ned, cab) på alle etasjer
5  * og oppdaterer tilstandsdata og knappelys.
6  * Ignorerer knappetrykk hvis stoppknappen er trykket eller hindring er
   aktiv.
7  *
8  * @param data Peker til heisens tilstandsdata
9  */
10 void checkForInput(state_data *data);
11
12 /**
13  * @brief Slår av alle knappelys for en spesifikk etasje
14  *
15  * Brukes når en etasje betjenes for å nullstille bestillinger og slå
   av lys.
16  *
17  * @param data Peker til heisens tilstandsdata
18  * @param floor Etasjen hvor knappelys skal slås av
19  */
20 void turnOffButtonLamps(state_data *data, int floor);
```

A.1.4 Oppstartsmodul (startup.h)

Modulen håndterer initialisering av heisen.

Listing A.4: Hovedfunksjoner i oppstartsmodulen

```
1 /**
2  * @brief Initialiserer heisen ved oppstart ved å kjøre til en definert
   posisjon
3  *
4  * Ignorerer alle bestillinger og stoppknappen under initialisering.
5  * Kjører heisen til nærmeste etasje hvis den er mellom etasjer.
6  *
7  * @return 1 når initialiseringen er fullført
```

```
8  */
9  int elevator_startup();
10
11 /**
12  * @brief Slår av alle knappelys
13  *
14  * Nullstiller alle knappelys ved systemoppstart.
15  */
16 void clear_buttons();
```

A.1.5 Grensesnitt mot heismaskinvare (elevio.h)

Modulen inneholder funksjoner for kommunikasjon med heismaskinvaren.

Listing A.5: Hovedfunksjoner i maskinvaregrensesnittet

```
1  /**
2   * @brief Initialiserer heismaskinvaren
3   *
4   * Må kalles en gang før andre funksjoner benyttes.
5   */
6  void elevio_init(void);
7
8  /**
9   * @brief Setter motorens retning
10  *
11  * @param dirn Ønsket retning (DIRN_UP, DIRN_DOWN, DIRN_STOP)
12  */
13  void elevio_motorDirection(MotorDirection dirn);
14
15  /**
16  * @brief Setter lampen for en spesifikk knapp
17  *
18  * @param floor Etasjen (0-indeksert)
19  * @param button Knappetypen
20  * @param value 1 for å slå på, 0 for å slå av
21  */
22  void elevio_buttonLamp(int floor, ButtonType button, int value);
23
24  /**
25  * @brief Setter etasjeindikatoren
26  *
27  * @param floor Etasjen som skal vises (0-indeksert)
28  */
29  void elevio_floorIndicator(int floor);
30
31  /**
32  * @brief Kontrollerer døråpningslampen
33  *
34  * @param value 1 for å slå på, 0 for å slå av
35  */
36  void elevio_doorOpenLamp(int value);
37
38  /**
39  * @brief Kontrollerer stoppknapplampen
40  *
41  * @param value 1 for å slå på, 0 for å slå av
42  */
```



```
43 void elevio_stopLamp(int value);
44
45 /**
46  * @brief Leser statusen til en ringeknapp
47  *
48  * @param floor Etasjen (0-indeksert)
49  * @param button Knappetypen
50  * @return 1 hvis knappen er trykket, 0 ellers
51  */
52 int elevio_callButton(int floor, ButtonType button);
53
54 /**
55  * @brief Leser etasjesensoren
56  *
57  * @return Nåværende etasje (0-indeksert), eller -1 hvis mellom etasjer
58  */
59 int elevio_floorSensor(void);
60
61 /**
62  * @brief Leser status på stoppknappen
63  *
64  * @return 1 hvis stoppknappen er trykket, 0 ellers
65  */
66 int elevio_stopButton(void);
67
68 /**
69  * @brief Leser status på hindringsbryteren
70  *
71  * @return 1 hvis hindring er aktiv, 0 ellers
72  */
73 int elevio_obstruction(void);
```

A.1.6 Hovedprogram (main.c)

Hovedprogrammet for heiskontrollsystemet.

Listing A.6: Hovedfunksjonen

```
1 /**
2  * @file main.c
3  * @brief Hovedprogram for heiskontrollsystemet
4  *
5  * Denne filen inneholder hovedløkken for heiskontrollsystemet.
6  * Den implementerer en tilstandsmaskin som håndterer de ulike
7  * heistilstandene og overganger mellom dem basert på inndata og
8  * hendelser.
9  *
10 * @author Erik Bohne & Lars Lien
11 */
12 /**
13  * @brief Hovedfunksjon for heiskontrollsystemet
14  *
15  * Initialiserer heismaskinvaren, utfører oppstartskalibrering,
16  * og kjører hovedkontrollløkken som implementerer
17  * heistilstandsmaskinen.
18  *
19  * @return 0 ved normal avslutning, -1 ved feil
```

```
19  */
20  int main(){
21      // Initialiser maskinvare
22      elevio_init();
23
24      // Initialiser tilstandsdatastruktur
25      state_data sData = {STANDING_STILL,
26                          {{0,0,0},{0,0,0},{0,0,0},{0,0,0}}, CURRENT_DIR_DOWN};
27
28      // Utfør heisoppstartskalibrering
29      elevator_startup();
30
31      // Hovedkontrollløkke
32      while(1){
33          // Oppdater etasjesensoravlesning
34          floor = elevio_floorSensor();
35          if(floor >= 0){
36              elevio_floorIndicator(floor);
37          }
38
39          // Sjekk først for hindringer - dette har høyest prioritet
40          if (check_obstruction(&sData, floor)) {
41              continue;
42          }
43
44          // Sjekk for knappeinndata
45          checkForInput(&sData);
46
47          // Tilstandsmaskinimplementasjon
48          switch (sData.state) {
49              case STANDING_STILL:
50                  standing_still(&sData, floor);
51                  break;
52              case DRIVING_UP:
53                  driving(&sData, floor, DIRN_UP);
54                  break;
55              case DRIVING_DOWN:
56                  driving(&sData, floor, DIRN_DOWN);
57                  break;
58              case DOOR_OPEN:
59                  door_open_state(&sData, floor);
60                  break;
61              default:
62                  return -1;
63                  break;
64          }
65
66          return 0;
67      }
```

A.2 Tilleggsdiagrammer

A.2.1 Tilstandsdiagram

Her er et detaljert tilstandsdiagram for heisstyringssystemet:

Figure A.1: Tilstandsdiagram for heisstyringssystemet

Diagrammet illustrerer følgende hovedtilstander og overganger:

- **DOOR_OPEN**: Heisen står med åpen dør
- **STANDING_STILL**: Heisen står stille og venter på bestillinger
- **DRIVING_UP**: Heisen beveger seg oppover
- **DRIVING_DOWN**: Heisen beveger seg nedover

A.2.2 Sekvensdiagram

Dette sekvensdiagrammet viser interaksjoner mellom systemkomponentene under typisk drift:

Figure A.2: Sekvensdiagram for typisk drift

Diagrammet illustrerer følgende hovedsekvenser:

- Registrering av bestillinger
- Beregning av retning
- Bevegelse mellom etasjer
- Håndtering av ankomst til etasje
- Døråpning og -lukking